```
# ====================================
# Step 1: Import Libraries
# ====================================
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
```

```
# ====================================
# Step 2: Load Dataset
# ====================================
# Please replace 'your_dataset.csv' with the actual path to your dataset file.
# For example, you can use '/content/sample_data/california_housing_train.csv' if you want to use a sample datas
data_df = pd.read_csv('/content/sample_data/california_housing_train.csv')
```

```
# ====================================
# Step 3: Basic Data Exploration
# ====================================
print(data_df.head())        # first few rows
print(data_df.info())        # dataset info
print(data_df.describe())    # statistical summary
```

```
0    -114.31     34.19                15.0       5612.0          1283.0
1    -114.47     34.40                19.0       7650.0          1901.0
2    -114.56     33.69                17.0        720.0           174.0
3    -114.57     33.64                14.0       1501.0           337.0
4    -114.57     33.57                20.0       1454.0           326.0

   population  households  median_income  median_house_value
0      1015.0       472.0         1.4936             66900.0
1      1129.0       463.0         1.8200             80100.0
2       333.0       117.0         1.6509             85700.0
3       515.0       226.0         3.1917             73400.0
4       624.0       262.0         1.9250             65500.0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17000 entries, 0 to 16999
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           17000 non-null  float64
 1   latitude            17000 non-null  float64
 2   housing_median_age  17000 non-null  float64
 3   total_rooms         17000 non-null  float64
 4   total_bedrooms      17000 non-null  float64
 5   population          17000 non-null  float64
 6   households          17000 non-null  float64
 7   median_income       17000 non-null  float64
 8   median_house_value  17000 non-null  float64
dtypes: float64(9)
memory usage: 1.2 MB
None
          longitude      latitude  housing_median_age   total_rooms  \
count  17000.000000  17000.000000        17000.000000  17000.000000
mean    -119.562108     35.625225           28.589353   2643.664412
std        2.005166      2.137340           12.586937   2179.947071
min     -124.350000     32.540000            1.000000      2.000000
25%     -121.790000     33.930000           18.000000   1462.000000
50%     -118.490000     34.250000           29.000000   2127.000000
75%     -118.000000     37.720000           37.000000   3151.250000
max     -114.310000     41.950000           52.000000  37937.000000

       total_bedrooms    population    households  median_income  \
```

```
mean       539.410824   1429.573941    501.221941       3.883578
std        421.499452   1147.852959    384.520841       1.908157
min          1.000000      3.000000      1.000000       0.499900
25%        297.000000    790.000000    282.000000       2.566375
50%        434.000000   1167.000000    409.000000       3.544600
75%        648.250000   1721.000000    605.250000       4.767000
max       6445.000000  35682.000000   6082.000000      15.000100

       median_house_value
count        17000.000000
mean        207300.912353
std         115983.764387
min          14999.000000
25%         119400.000000
50%         180400.000000
75%         265000.000000
max         500001.000000
```

```
# ====================================
# Step 4: Missing Values
# ====================================
print(data_df.isnull().sum())
```

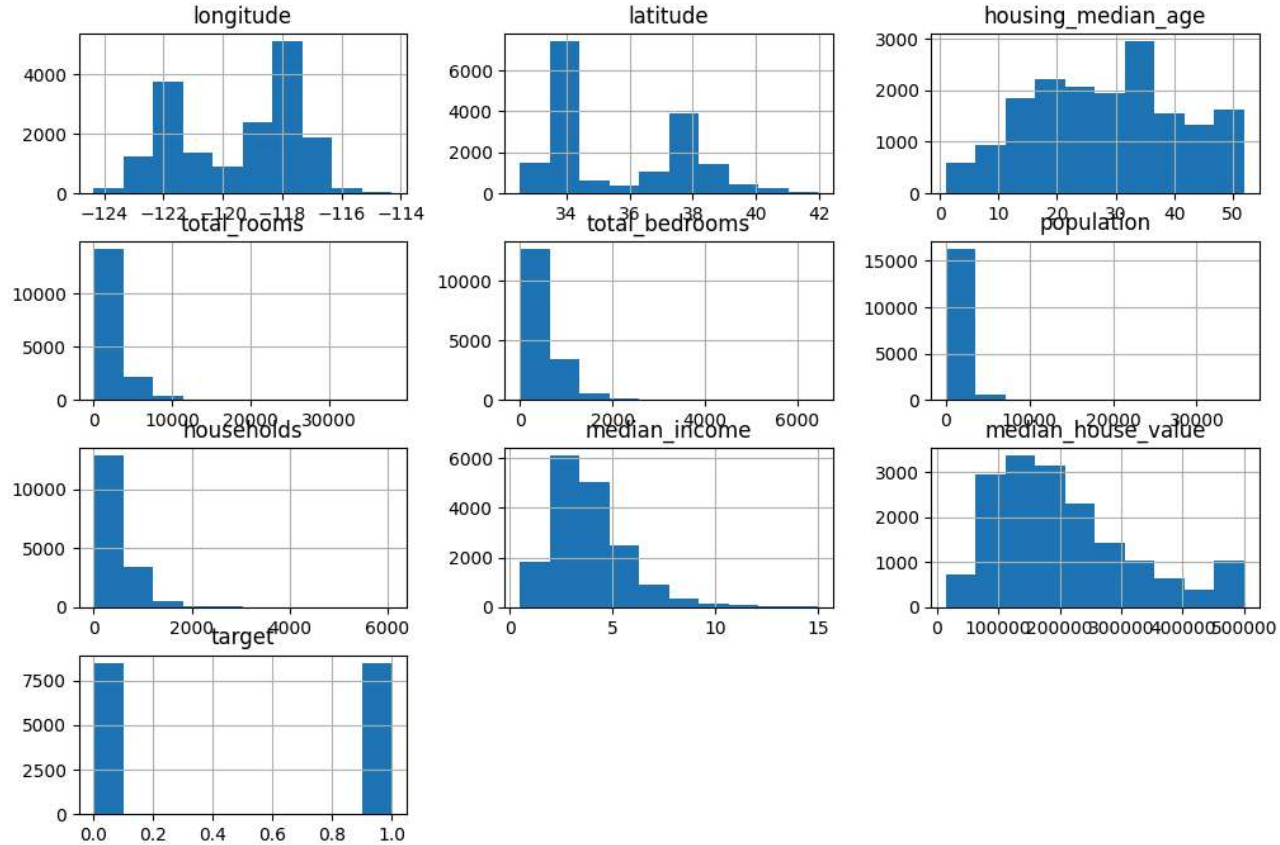
```
longitude             0
latitude              0
housing_median_age    0
total_rooms           0
total_bedrooms        0
population            0
households            0
median_income         0
median_house_value    0
dtype: int64
```

```
# ====================================
# Step 5: Class Distribution
# ====================================
# The 'target' column is not natively present in the dataset.
# It needs to be created before its distribution can be analyzed.
# Creating a binary 'target' column based on 'median_house_value' for classification example.
median_house_value_median = data_df['median_house_value'].median()
data_df['target'] = (data_df['median_house_value'] > median_house_value_median).astype(int)

print("New 'target' column created for class distribution analysis.")
print(data_df["target"].value_counts())
```

```
New 'target' column created for class distribution analysis.
target
0    8508
1    8492
Name: count, dtype: int64
```

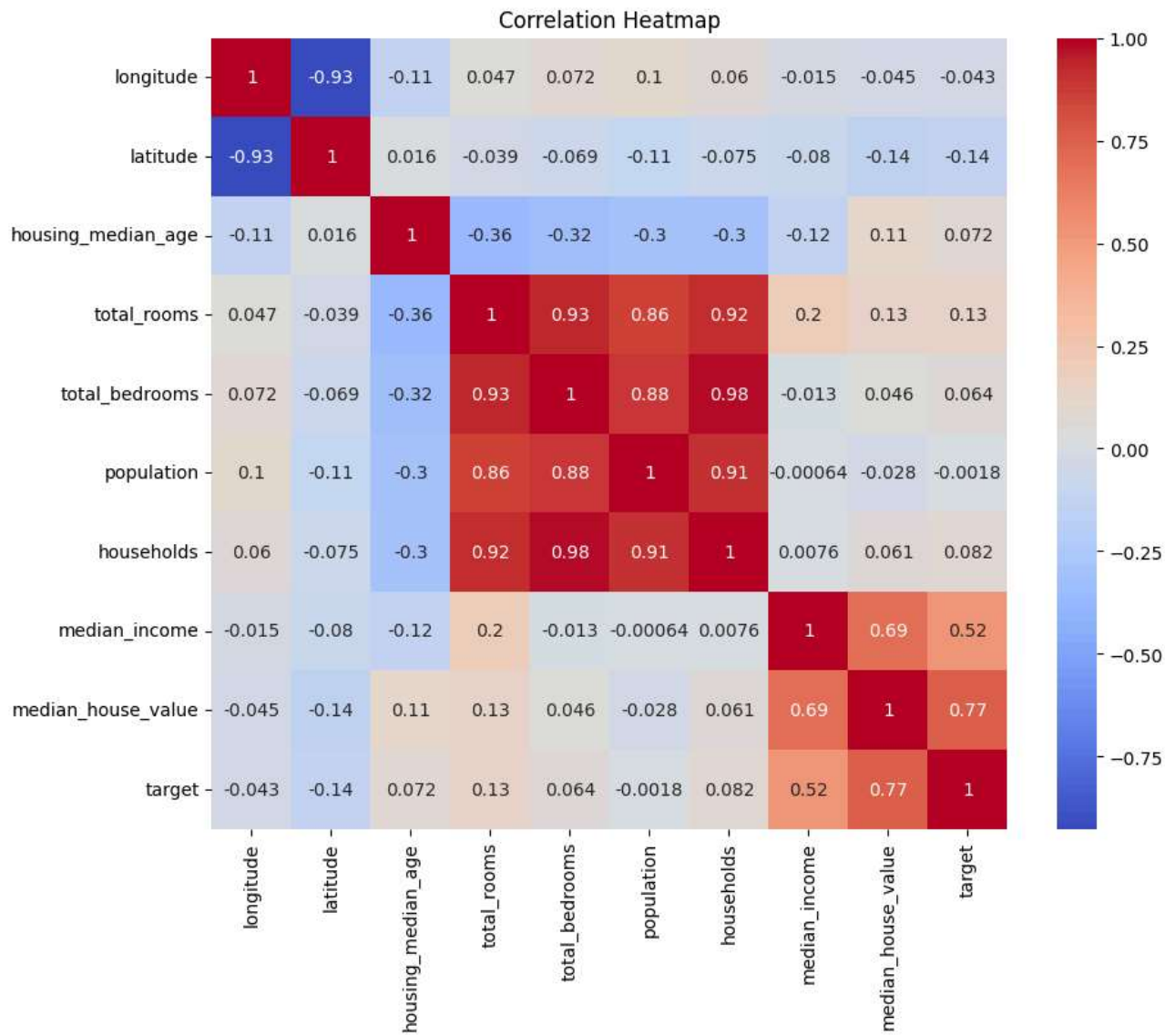
```
# ====================================
# Step 6: Histogram Plot
# ====================================
data_df.hist(figsize=(12, 8))
plt.show()
```




```
# ====================================
# Step 7: Correlation Heatmap
# ====================================
plt.figure(figsize=(10, 8))

sns.heatmap(
    data_df.corr(),
    annot=True,
    cmap="coolwarm"
)

plt.title("Correlation Heatmap")
plt.show()
```

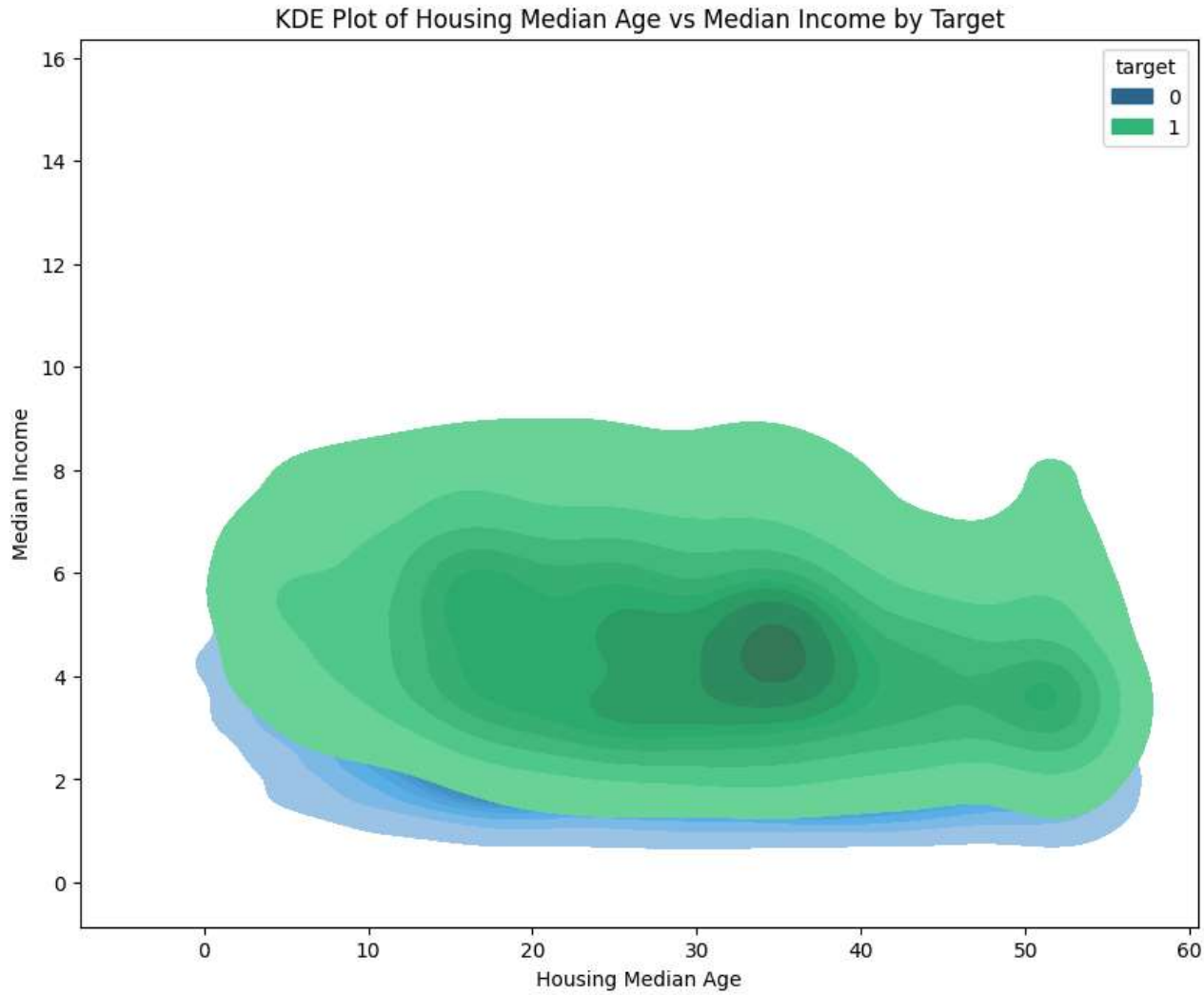



```
# ====================================
# Step 8: KDE Plot (uncommented and adapted for new target)
# ====================================
# This step assumes relevant numerical features exist.
# Using 'housing_median_age' and 'median_income' as example features.
plt.figure(figsize=(10, 8))

sns.kdeplot(
    x=data_df["housing_median_age"],
    y=data_df["median_income"],
    hue=data_df["target"], # using the newly created binary target column
    fill=True,
    palette="viridis" # different palette for clarity
)

plt.title("KDE Plot of Housing Median Age vs Median Income by Target")
plt.xlabel("Housing Median Age")
plt.ylabel("Median Income")

plt.show()
```




```
# ====================================
# Step 9: Feature & Target Split
# ====================================
# Define features (X) and target (y)
# Exclude 'median_house_value' and the original 'median_house_value' column from features
X = data_df.drop(["target", "median_house_value"], axis=1)
y = data_df["target"]
```

```
# ====================================
# Step 10: Train-Test Split
# ====================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# ====================================
# Step 11: Feature Scaling
# ====================================
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# ====================================
# Step 12: Train SVM Model
# ====================================
model = SVC(kernel="linear", random_state=42) # Added random_state for reproducibility
```

```
model.fit(X_train, y_train)
```

```
▸ SVC
```

```
# ====================================
# Step 13: Prediction & Evaluation
# ====================================
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.841764705882353
```

```
# ====================================
# Create a binary target for classification example
# ====================================
median_house_value_median = data_df['median_house_value'].median()
data_df['target'] = (data_df['median_house_value'] > median_house_value_median).astype(int)
print("New 'target' column created based on median_house_value.")
print(data_df['target'].value_counts())

# ====================================
# Step X: Pairplot Visualization
# ====================================
sns.pairplot(
    data_df,
    hue="target",        # using the newly created binary target column
    diag_kind="kde",     # diagonal plots as KDE
    palette="coolwarm"   # color palette
)

plt.suptitle("Pairplot of Features vs Target", y=1.02)
plt.show()
```

```
New 'target' column created based on median_house_value.
target
0    8508
1    8492
Name: count, dtype: int64
```